

Full Stack Development

What is Full Stack Development?

Full stack development refers to the practice of working on both the frontend (what the user sees) and the backend (the server, logic, and database) of a web application. A full stack developer is capable of building a complete web application from start to finish.

The term "stack" refers to the set of technologies used together to build a working web application. In this course we work with HTML, CSS, and Bootstrap on the frontend, PHP on the backend, and MySQL / MariaDB as the database.

The Three Layers of a Web Application

Every web application is built from three distinct layers, each with a specific role:

Layer	Role	Technologies (This Course)
Frontend (Client Side)	Everything the user sees and interacts with in the browser.	HTML, CSS, Bootstrap, JavaScript
Backend (Server Side)	Handles HTTP requests, processes business logic, and communicates with the database.	PHP
Database (Data Layer)	Stores and manages all persistent data such as users and records.	MySQL, MariaDB

Frontend

The frontend is everything rendered in the browser. It is built using:

- **HTML** : structures the content (headings, paragraphs, forms, tables).
- **CSS** : controls the visual appearance (colours, fonts, layout, spacing).
- **Bootstrap** : a CSS framework that provides ready-made responsive components.
- **JavaScript** : adds interactivity such as form validation and dynamic content updates.

Backend - PHP

The backend handles all server-side logic. In this course we use PHP. PHP runs on the web server, processes incoming HTTP requests from the browser, interacts with the database, and returns a response (an HTML page or data).

```
<?php
// Connect to the database
$conn = new mysqli('localhost', 'root', '', 'mydb');

// Run a query
$result = $conn->query('SELECT * FROM users');

// Output each row as HTML
while ($row = $result->fetch_assoc()) {
    echo '<p>' . $row['name'] . '</p>';
}
?>
```

Key PHP responsibilities in a full stack application:

- **Receive form data** : handle POST/GET submissions from HTML forms via `$_POST` / `$_GET`.

- **Run database queries** : read or write data using SQL through MySQLi or PDO.
- **Session management** : maintain user login state across pages using \$_SESSION.
- **Generate HTML output** : dynamically build and return complete HTML pages.

Database - MySQL & MariaDB

The database layer stores all application data. In this course we use MySQL and MariaDB. MariaDB is a community fork of MySQL and is fully compatible the same SQL syntax works for both.

```
-- Create a users table
CREATE TABLE users (
  id    INT AUTO_INCREMENT PRIMARY KEY,
  name  VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  created TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert a record
INSERT INTO users (name, email)
VALUES ('Raunak Poudel', 'raunakpoudel0@gmail.com');

-- Retrieve all users
SELECT * FROM users;

-- Update a record
UPDATE users SET name = 'R. Poudel' WHERE id = 1;

-- Delete a record
DELETE FROM users WHERE id = 1;
```

How a Web Request Works (End-to-End)

When a user submits a form on a PHP-powered website, the following process takes place:

1. The user fills in an HTML form in the browser and clicks Submit.
2. The browser sends an HTTP POST request to a PHP file on the server.
3. PHP receives the form data via \$_POST and validates it.
4. PHP runs a SQL query against the MySQL / MariaDB database.
5. The database processes the query and returns a result to PHP.
6. PHP generates an HTML response and sends it back to the browser.
7. The browser displays the updated page to the user.

Technology Stack Summary

Area	Technology	Purpose
Frontend	HTML5	Structure and content of web pages
Frontend	CSS3	Styling and layout
Frontend	Bootstrap 5	Responsive UI components and grid
Frontend	JavaScript	Interactivity and DOM manipulation
Backend	PHP	Server-side logic and form handling
Database	MySQL/MariaDB	Storing and querying application data